

Enginyeria del Programari de Components i Sistemes Distribuïts

PROVA D'AVALUACIÓ CONTINUADA – PAC 1 SOLUCIÓ

Presentació

Aquesta primera PAC serveix com introducció a les architectures de programari pel desenvolupament d'aplicacions distribuïdes, on es treballen els fonaments, tècniques, i habilitats aplicables a la definició moderna d'arquitectures del programari, i s'aprofundeix en la selecció de l'estil arquitectònic més adient segons la tipologia de l'aplicació a desenvolupar.

Per elaborar l'activitat es formulen quatre exercicis: el primer avaluarà els aspectes fonamentals de la definició d'una arquitectura. En el segon i tercer exercicis caldrà decidir i raonar l'estil arquitectònic (o combinació d'estils) més adient segons els requeriments plantejats en un cas d'estudi. Finalment, el darrer exercici avaluarà certs conceptes rellevants de les architectures distribuïdes.

L'abast de la PAC 1 són els continguts del llibre "*Fundamentals of Software Architecture*" descrits en els Recursos d'aprenentatge d'aquesta activitat.

Competències

En aquesta PAC 1 es treballa la següent competència del Grau en Enginyeria Informàtica:

- Proposar i avaluar diferents alternatives tecnològiques per resoldre un problema concret.

Objectius

Els objectius de la PAC 1 són:

- Explicar els fonaments d'arquitectura de programari i les característiques més rellevants.
- Exposar arguments sobre els punts forts i punts febles del diferents estils arquitectònics, i com aquests influeixen en la decisió de l'arquitectura més adients per una determinada aplicació.
- Situar els nous paradigmes emergents de l'enginyeria i l'arquitectura del programari.

Descripció de la PAC a realitzar

Exercici 1 (2 punts)

Per les característiques d'arquitectura presentades en la taula següent, escolliu un estil arquitectònic que maximitzi i un altre que minimitzi la característica. En cas de tenir més d'un estil possible, escolliu només un com a candidat. Cal justificar cada elecció presa.

Teniu com exemple la característica d'arquitectura **Fault tolerance**. Completeu la resta de la taula.

	<i>Estil arquitectònic que maximitza la característica</i>	<i>Estil arquitectònic que minimitza la característica</i>
<i>Deployability</i>		
<i>Elasticity</i>		
<i>Evolutionary</i>		
<i>Fault tolerance</i>	Event Driven: Obté una màxima tolerància a fallides emprant <i>event processors</i> altament desacoblats i asíncrons, que proporcionen consistència i processament eventual dels <i>event workflows</i> .	Pipeline i Layered: No proporcionen tolerància a fallides per la seva naturalesa de desplegament monolític, i la manca de modularitat. En cas de fallida d'una part del sistema, sovint es veu afectat tot el sistema complet. Microkernel: Els connectors depenen del sistema principal, per tant qualsevol incidència pot afectar mútuament.
<i>Modularity</i>		
<i>Overall cost</i>		
<i>Performance</i>		
<i>Reliability</i>		
<i>Scalability</i>		
<i>Simplicity</i>		
<i>Testability</i>		

Solució:

Nota: En la solució s'enumeren els estils arquitectònics que maximitzen i minimitzen una determinada característica. Tal com indicava l'enunciat, identificant un d'ells seria suficient.

	<i>Estil arquitectònic que maximitza la característica</i>	<i>Estil arquitectònic que minimitza la característica</i>
<i>Deployability</i>	Microservices: El desplegament de microserveis permet unitats d'implementació molt petites i desacoblades, amb un desplegament de forma aïllada	Layered: Un simple canvi implica tornar a desplegar tota l'aplicació Orchestration-Driven Service-Oriented: Poc suport a ser desplegat de forma fàcil
<i>Elasticity</i>	Microservices: En una arquitectura basada en automatització i integració intel·ligent de les operacions, es pot incloure també suport a l'elasticitat.	Layered: Molt poca elasticitat degut a la naturalesa monolítica dels desplegaments, i la manca de modularitat. Microkernel: Molt poca elasticitat doncs els connectors depenen de la infraestructura del mòdul principal.
<i>Evolutionary</i>	Event-Driven: Excel·lent per afegir noves funcionalitats, tant processadors d'esdeveniments existents com creant nous. Microservices: Afavoreix un elevat desacoblament a nivell incremental i canvi evolutiu. Amb unitats d'implementació petites i desacoblades, l'arquitectura disposa d'una estructura que permet un ritme de canvis més ràpid.	Layered: Molt dolenta, per l'elevat acoblament i poca modularitat. Orchestration-Driven Service-Oriented: Molt dolenta per l'acoblament amb el punt central d'orquestració.
<i>Fault tolerance</i>	Event Driven:	Pipeline i Layered:

	Obté una màxima tolerància a fallides emprant <i>event processors</i> altament desacoblats i asíncrons, que proporcionen consistència i processament eventual dels <i>event workflows</i>.	No proporcionen tolerància a fallides per la seva naturalesa de desplegament monolític, i la manca de modularitat. En cas de fallida d'una part del sistema, sovint es veu afectat tot el sistema complet. Microkernel: Els connectors depenen del sistema principal, per tant qualsevol incidència pot afectar mútuament.
<i>Modularity</i>	Microservices: Arquitectura modular amb unitats d'implementació petites i desacoblades.	Layered: Arquitectura que no compleix modularitat.
<i>Overall cost</i>	Layered o Pipeline: Pe la seva naturalesa monolítica, no tenen les complexitats de construcció i manteniment associades a una arquitectura monolítica, i en aquest sentit tenen un cost relativament més baix. Microkernel: Cost baix davant la facilitat en crear o eliminar connectors segons les necessitats.	Orchestration-Driven Service-Oriented: Cost molt elevat degut a la naturalesa de la seva participació principalment tècnica, i la complexitat en gestionar l'acoblament amb el motor d'orquestració.
<i>Performance</i>	Event-Driven: Eficiència en el rendiment gràcies a les comunicacions asíncrones combinades amb el processament en paral·lel dels esdeveniments.	Layered: La seva naturalesa monolítica complica les optimitzacions en el rendiment, per la baixa escalabilitat i modularitat de l'arquitectura, i la manca de processament en paral·lel de la lògica de negoci, i el possible anti-patró d'excessiva dependències de crides entre capes.
<i>Reliability</i>	Service-Based:	Orchestration-Driven Service-Oriented:

	Bona fiabilitat per la granularitat dels serveis i requerir menys operacions de xarxa. Microservices: Arquitectura que permet tècniques de fail-over dels components davant fallida.	Baixa, degut a la quantitat de components que han de ser executats per cada acció sobre el sistema.
<i>Scalability</i>	Event-Driven: Escalabilitat elevada gràcies a tècniques de balancejament programàtic dels processadors d'esdeveniments.. Microservices: Els sistemes actuals més escalables han estat implementats amb aquesta arquitectura.	Layered i Pipeline: Baixa escalabilitat per la naturalesa monolítica dels desplegaments. Malgrat és possible escalar certes funcionalitats en un monòlit, l'esforç és elevat i les tècniques són complexes. Microkernel: Poca escalabilitat perquè els connectors depenen de la infraestructura del mòdul principal.
<i>Simplicity</i>	Layered i Pipeline: No tenen les complexitats associades als estils d'arquitectura distribuïda, per tant són més simples i fàcils d'entendre.	Event-Driven: Arquitectura molt complexa en orquestrar els esdeveniments. Orchestration-Driven Service-Oriented o Microservices: Molt complexa per la quantitat de components d'infraestructura de caire no funcional que intervenen.
<i>Testability</i>	Microservices: Facilitat de testear microserveis al separar l'arquitectura en components més petits i independents.	Orchestration-Driven Service-Oriented: Arquitectura que planteja una elevada complexitat en preparar escenaris de proves.

Exercici 2 (3 punts)

Un negoci familiar de restauració que hostatjava tres pizzeries en la perifèria de la ciutat de Girona, va decidir incorporar el repartiment a domicili.

Per tant, a banda de l'atenció telefònica, varen crear una aplicació web molt senzilla, amb la carta de productes disponibles permetent als clients fer les seves comandes. Van aconseguir posar en marxa l'aplicació en un temps molt ràpid, superant amb èxit les seves expectatives de negoci.

L'empresa es va anar expandint amb l'obertura de noves pizzeries. A dia d'avui, disposen de desenes de restaurants i una base de dades de client molt gran. L'èxit ha estat tan gran, que fins i tot, es plantegen convertir-se en una franquícia i donar el salt a tot el mercat nacional.

Però degut a aquest creixement tant ràpid, estan començant a sorgir certs problemes en l'aplicació actual que cal plantejar com a reptes a resoldre. Els enumeren breument:

- En hora punta, el nombre d'usuaris fent comandes és molt elevat, i l'aplicació web respon molt lentament doncs els servidors van molt saturats. L'alta concurrència comença a ser un problema molt greu per a la plataforma.
- En aquests períodes d'ús molt intensiu, la infraestructura actual ja no és suficient, i la plataforma finalment es cau. Cal tenir en compte que no sembla una proposta encertada ampliar la infraestructura únicament per atendre aquests moments d'alta demanda, doncs la resta de temps tindríem aquest extra de recursos d'infraestructura sense utilitzar.
- Cada cop sembla més clar que els requeriments tecnològics no són homogenis per tota l'aplicació.
- S'han trobat errors de regressió que han deixat la plataforma fora de servei durant hores després d'una actualització de versió, amb la conseqüent pèrdua econòmica per a l'empresa. Això és degut a la manca de tests automatitzats que assegurin que les noves release no afecten el comportament de l'aplicació i no es trenca res.
- El plantejament d'un model de negoci en franquícia requereix un coneixement més específic tant a nivell funcional com tècnic. Ja no és factible que el petit equip de desenvolupament actual domini tots aquests conceptes. Per exemple, s'incorporen nous processos o integracions complexes que es poden resoldre millor amb tècniques de programació funcional o bases de dades noSQL, mentre que els processos de facturació, tot i que també són complexos, requereixen ser implementats amb tecnologies més tradicionals.
- Per guanyar ser més competitius, cal desenvolupar noves funcionalitats (per exemple: promocions puntuals) i desplegar-les a producció molt ràpidament. Idealment, una funcionalitat hauria de promocionar fins a l'entorn de producció de manera segura sense massa intervenció manual, doncs aquest cicle podria repetir-se diverses vegades. Això implica que els desplegaments de funcionalitat haurien de ser independents, cadascun seguint el seu propi cicle de vida.

Quin dels estils arquitectònics presentats en aquesta primera activitat (*Fundamentals of Software Architecture*, Capítol 2: *Architecture Styles*) considereu que encaixaria millor en aquest escenari?

Justifiqueu la vostra resposta comparant els requeriments i restriccions del projecte, amb els punts a favor i en contra de l'estil arquitectònic que heu escollit.

Solució:

L'estil arquitectònic requerit per aquesta evolució del projecte seria el basat en microserveis (**Microservices Architecture**). Alguns dels motius es detallen a continuació:

- Una arquitectura basada en microserveis és altament escalable. Desplegada sobre una plataforma adequada al Cloud, permetrà solucionar els problemes de rendiment i de pics puntuals d'utilització amb un cost raonable, emprant tècniques d'autoescalament.
- Una arquitectura basada en microserveis permet escollir la millor tecnologia per cada servei i que aquests col·laborin entre ells.
- Els microserveis poden exposar dominis funcionals altament especialitzats, malgrat considerar-se com una "caixa negra" tecnològicament per a la resta del sistema. Això solventa el problema d'especialització referenciat en l'enunciat, al passar d'una separació purament tecnològica a una separació per "conceptes de domini" pròpia d'aquest estil d'arquitectura.
- Un microservei evoluciona i es pot desplegar de forma independent a la resta de components del sistema. Això permet desenvolupar noves funcionalitats i posar-les a producció molt ràpidament.
- La "testeabilitat" és una bases d'aquesta arquitectura, on cada microservei disposa de la seva bateria de tests automatitzats que poden prevenir la introducció d'errors de regressió en els desplegaments.

Per altre banda, alguns dels punts febles d'aquest estil són: la complexitat i el rendiment referent a la naturalesa distribuïda i els costos de desenvolupament.

També és rellevant que una implantació eficient de microserveis requereix una cultura Devops madura i metodologia de desenvolupament àgil.

Trobareu al capítol 17: *Microservices Architecture* del libro *Fundamentals of Software Architecture* una explicació més extensa i detallada de l'arquitectura proposada, i el seu possible encaix segons els requisits del projecte.

Exercici 3 (3 punts)

La botiga de material audiovisual especialitzada en articles per rodatges de pel·lícules i sessions professionals de fotografia, "**Photo&Film4You**", pretén desenvolupar una primera versió del servei de lloguer especificat en el cas d'estudi adjunt en aquest enunciat.

El responsable de "**Photo&Film4You**" vol fer una primera versió per avaluar si el lloguer per internet dels seus equipaments i venda pot ser rentable, o no tindria massa acceptació.

A tal efecte, pretén crear una aplicació web senzilla amb la llista d'equips disponibles i una funcionalitat de lloguer per als clients. Per implementar aquesta aplicació s'ha contractat una petita empresa que disposa d'un equip de desenvolupament propi amb dos programadors Java, un amb coneixements en bases de dades relacionals, i un altre amb coneixements en capa de presentació. El membre més experimentat de l'equip, actuarà també com arquitecte del projecte.

El desplegament de l'aplicació serà sobre uns servidor d'aplicacions JBoss, i les dades quedaran en una base de dades relacional MySQL. Tant el servidor d'aplicacions com la base de dades estaran instal·lades en infraestructures "on-premises".

Per últim, cal tenir en compte que es prioritza tenir una primera versió funcionant ràpidament i amb un cost baix, sense considerar encara possibles problemes futurs de rendiment, mantenibilitat o modularitat. Ara mateix, l'objectiu del projecte és avaluar només si el model de negoci seria rentable.

De nou, quin estil arquitectònic dels presentats en el Capítol 2: *Architecture Styles* considereu que encaixa millor amb la naturalesa d'aquest projecte?

Raoneu també la resposta comparant els requeriments i restriccions del projecte a desenvolupar, amb els pros i contres de l'estil arquitectònic que trieu.

Solució:

L'estil arquitectònic més adient per desenvolupar aquest exercici seria per capes (**Layered Architecture Style**). Alguns dels motius serien:

- Segons l'enunciat, cal construir una aplicació web senzilla amb una tecnologia coneguda, cost baix i disponible ràpidament. L'estil en capes està principalment enfocat a resoldre aquest tipus de requeriments: "*This style of architecture is the de facto standard for most applications, primarily because of its simplicity, familiarity, and low cost*".
- Amb un equip de desenvolupament que no són experts en el domini, la separació per capes seria de tipus tecnològic, que és precisament la que s'implementa en un estil d'arquitectura per capes: "*The layered architecture is a technically partitioned architecture (as opposed to a domain-partitioned architecture)*".
- Aquesta arquitectura encaixa perfectament amb una topologia basada en una interfície web, amb un servidor d'aplicacions per la capa de negoci un servidor de base de dades a banda, amb un desplegament de tipus "on-premise".

Algun dels punts febles d'aquest estil són la modularitat, escalabilitat, rendiment i mantenibilitat. Malgrat segons els requeriments de l'enunciat aquestes característiques són poc rellevants en la situació actual del projecte.

Podeu consultar el capítol 10. *Layered Architecture Style* del llibre *Fundamentals of Software Architecture* per una explicació més detallada de l'arquitectura proposada per aquest exercici.

Exercici 4 (2 punts)

Responen breument a les següents preguntes sobre els conceptes vistos al llibre "*Fundamentals of Software Architecture*".

4.1 (0.5 punts)

Indiqueu sobre quin prisma o dimensió principal que conforma la definició d'una arquitectura (estructura, característica, principi de disseny, o decisió) s'emmarquen les següents decisions:

Solució:

- El sistema a desenvolupar serà un híbrid entre una arquitectura basada en capes i una arquitectura microkernel. **Estructura**
- El sistema a desenvolupar requereix recuperació davant qualsevol fallida en comunicacions amb sistemes externs. **Característica**
- El sistema a desenvolupar no ha de permetre un accés al model de dades des de la capa de presentació de l'usuari. **Decisió**
- En la mesura que sigui possible, s'utilitzaran tècniques de comunicació asíncrones entre processos, que no siguin bloquejants. **Principi de disseny**
- El backend del sistema a desenvolupar únicament serà accessible mitjançant una api REST. **Decisió**
- El sistema ha de ser testeable i segur. **Característica**

4.2 (0.5 punts)

Quines d'aquestes tasques considereu que són responsabilitat d'un arquitecte, i quines són responsabilitat de l'equip de desenvolupament?

Solució:

- Crear els tests d'integració de l'aplicació. **Equip de desenvolupament**
- Analitzar les alternatives dels diferents proveïdors de Cloud per disposar d'autoescalament. **Arquitecte**
- Crear el diagrama de classes d'un microservei de catàleg per una aplicació de comerç online. **Equip de desenvolupament**

- Decidir la tecnologia per implementar la seguretat global d'un projecte. **Arquitecte**
- Crear el prototipus d'interfície d'usuari segons les directrius dels experts d'usabilitat. **Equip de desenvolupament**
- Decidir l'estratègia de desplegament del sistema. **Arquitecte**

Podeu consultar el [capítol 2. "Architectural Thinking - Architecture Versus Design"](#) del llibre *"Fundamentals of Software Architecture"*.

4.3 (0.5 punts)

Indiqueu si les següents afirmacions són certes o falses:

Solució:

- Un arquitecte no codifica. Aquestes tasques s'adrecen únicament als desenvolupadors. **Fals**
- Es desitjable que un arquitecte conegui el model de negoci. **Cert**
- Una arquitectura sempre tindrà punts forts i punts febles (sovint, en contraposició entre ells) **Cert**
- Sempre és preferible una arquitectura de microserveis respecte una arquitectura per capes **Fals**
- És preferible fer un particionament per domini davant un particionament purament tècnic. **Fals**

4.4 (0.5 punts)

Escolliu dues expectatives que considereu associades al rol d'Arquitecte, debatudes en el fòrum de l'Activitat 1, i poseu un exemple de tasca concreta per cadascuna d'elles. No utilitzeu els exemples que ja figuren al llibre *Fundamentals of Software Architecture*.

Solució:

- **Analitzar contínuament l'arquitectura.** *Avaluar les alternatives de desplegament al núvol, i determinar si és possible executar una migració en aquest sentit.*
- **Tenir cert coneixement i domini del negoci:** *Per exemple, en el negoci bancari, conèixer els diferents productes que aquest ofereix*

Podeu revisar el capítol 1: *Introduction* del llibre *Fundamentals of Software Architecture*.

Recursos

Recursos bàsics

- Llibre "Fundamentals of Software Architecture" (Richards & Ford): Chapter 1. Introduction.
- Llibre "Fundamentals of Software Architecture" (Richards & Ford): I Foundations (Chapters 2, 4, 5 i 8).
- Llibre "Fundamentals of Software Architecture" (Richards & Ford): II Architecture Styles (Chapters 9 - 14, 16 - 18).

Criteris d'avaluació

- La PAC s'ha de resoldre de forma **estrictament individual**. En aquesta activitat **no està permès l'ús d'eines d'intel·ligència artificial**. En cas de detectar irregularitats, es penalitzarà l'activitat amb una D com a nota. **Això inclou la reutilització de solucions d'aquesta mateixa prova de semestres anteriors**. Al pla docent, i al [web sobre integritat acadèmica i plagi de la UOC](#), trobareu informació sobre què es considera conducta irregular en l'avaluació i les conseqüències que pot tenir.
- El pes de cada exercici està indicat a l'enunciat.
- És necessari justificar la solució a cadascun dels exercicis. Es valorarà tant les vostres decisions com la justificació donada a tal efecte.

Format i data de lliurament

Cal lliurar un únic document PDF amb les respostes a tots els exercicis. El nom de l'arxiu ha de ser: *CognomsNom_EPCSD_PAC1.pdf*.

Aquest document es lliurarà a l'espai "Lliurament de la PAC1" de l'aula de l'aula, abans de les **23:59 hores del dia 17 de març de 2025**. No s'acceptaran lliuraments fora de termini.

Annex

Criteris d'avaluació

	Excel·lent (10 punts)	Notable (7 punts)	Suficient (5 punts)	Insuficient (2,5 punts)	Sense resposta / Tot malament (0 punts)
Ex. 1 (2 p.)	Elecció correcta d'estils per 9 o més característiques i explicacions correctes (2)	Elecció correcta d'estils per entre 7 i 8 característiques sense explicacions o amb explicacions incorrectes (1,5)	Elecció correcta d'estils per entre 5 i 7 característiques i explicacions correctes (1)	Elecció correcta d'estils per menys de 5 característiques i explicacions correctes (0,5)	0
Ex. 2 (3 p.)	Elecció arquitectònica correcta amb raonament correcte i adequat en funció dels raonaments i restriccions de l'enunciat (3)	Elecció arquitectònica correcta, però raonament incorrecte o no adequat en funció dels raonaments i restriccions de l'enunciat (2)	Elecció arquitectònica incorrecte, però raonament adequat en funció dels raonaments i restriccions de l'enunciat (1,5)	Elecció arquitectònica correcta, però no raonada (0,75)	0
Ex. 3 (3 p.)	Elecció arquitectònica correcta amb raonament correcte i adequat en funció dels raonaments i restriccions de l'enunciat (3)	Elecció arquitectònica correcta, però raonament incorrecte o no adequat en funció dels raonaments i restriccions de l'enunciat (2)	Elecció arquitectònica incorrecte, però raonament adequat en funció dels raonaments i restriccions de l'enunciat (1,5)	Elecció arquitectònica correcta, però no raonada (0,75)	0
Ex. 4 (2 p.)	4 respostes correctes i ben raonades (2)	3 respostes correctes i ben raonades, i 1 resposta incorrecte o no raonada (1,5)	2 respostes correctes i ben raonades, i/o 2 respostes incorrectes o no raonades (1)	1 resposta correcta i ben raonada, i 3 respostes incorrectes i/o no raonades (0,5)	0